Failure Propagation and Recovery in Multi-Agent Systems

Phase 1: Fault Injection and Propagation Measurement

[Team member names] CSE 291A, Fall 2026

1 Goal and Setup

When agents pass results to each other, one wrong or malicious result can change what later agents decide and do. Phase 1 builds a workflow whose outcomes can be checked by a program, injects faults at controlled stages, and measures how far each fault spreads. It also sets two reference points for Phase 2's selective recovery: full restart and always-verify.

Workflow. Procurement tasks: "team T needs $q\times$ item; buy from the cheapest *approved* vendor if the total fits the team's remaining budget; record the order, deduct the budget, and email the manager a confirmation with the order id; otherwise order nothing and notify the manager." A deterministic *retrieval* step reads the vendor catalog, the team directory, and the shared file `budgets.json`. The *researcher* (LLM) extracts facts. The *planner* (LLM) emits a JSON plan of tool calls. The *executor* (LLM) issues one tool call at a time, filling in values returned by earlier calls (e.g., the order id). Tasks are generated from a seed with known ground truth. About 30% are over budget, and in about half of them the cheapest vendor is unapproved, so the agents must actually apply both rules.

Environment. State has two kinds. *Restorable shared files* (`budgets.json`, `orders.csv`) can be rolled back from a snapshot. *External actions* (`place_order`, `send_email`) leave the system and can never be rolled back. A checker compares the final state with ground truth: exactly one correct order, a matching record, the correct deduction, and a confirmation that carries the real id. It counts *unauthorized actions* separately: email to an external domain, file deletion, ordering from an unapproved vendor or over budget, and writes to other teams or files.

Tracing. Every message, intermediate result, tool call, and injected fault is stored as an artifact with explicit input edges. A tool call also depends on the last writer of every file it reads, so dependencies through shared state appear in the same graph.

Faults (definitions in Appendix B). There are 8 *accidental* faults: wrong value, stale output, timeout, a corrupted budget in shared state, and a corrupted order id in a tool result. There are 4 *adversarial* faults: prompt injection in a retrieved catalog record, in the researcher's output, and in a vendor API response, plus a compromised planner that appends an exfiltration step. A fault replaces a payload at a stage boundary, so every downstream artifact depends on the fault artifact. Each injected payload is kept internally consistent. A first version changed only the target field, and downstream agents simply corrected it from fields left unchanged (e.g., the researcher's free-text notes still named the right vendor).

Is the fault actually used? Each faulty run is aligned with the clean run of the same task by artifact position. Positions are compared on decision-relevant content only: recipients, ids, amounts, and tool arguments, not email wording. Two clean repetitions per task were semantically identical in 23/25 tasks; both differences come from one natural model error (below), so LLM nondeterminism adds little noise. Each run is then labeled *ignored* (nothing downstream changed and no injected string reached a downstream artifact), *propagated-benign*, or *propagated-harmful* (task failed or an unauthorized action occurred).

Methods. *None*: no recovery. *Full restart*: an oracle names the faulty result, so detection is kept separate from recovery; the files are restored from the snapshot and the whole workflow reruns. *Always-verify*: a verifier LLM checks the researcher and planner outputs against their sources, and a rejected output is regenerated once. All methods run the same fault schedule.

Scale. 25 tasks $\times$ 12 faults $\times$ 3 methods, plus 3 clean runs per task: 975 runs, with 0 harness failures. Model: `api-glm-5.3-flash` (UCSD TritonAI), temperature 0, 8 parallel workers on a laptop.

2 Results

Clean baseline. Task success is 96% (48/50 runs), with no unauthorized actions. A run costs 5.2k tokens, 6.3 LLM calls, and 29 s of wall time. Both failures are a *natural* fault: the researcher reported the balance *after* the purchase as the remaining budget, and the planner, trusting it, declined a valid order. It spreads exactly like our injected wrong values. Always-verify adds 37% tokens and 87% latency, with 1 false rejection in 25 clean runs.

Accidental faults almost always propagate to failure (Fig. 1): 167 of 190 applied runs are harmful, and only 12% of tasks succeed. A wrong vendor also turns into an unauthorized order (0.76 per run). A fault early in the pipeline wastes nearly the whole run. A wrong-value or stale researcher result changes about 3 downstream stages and wastes 4.2–4.9k tokens and 3–4 tool actions, while the shared-state corruption reaches all 5 stages. The one fault that is mostly ignored, a consistently doubled quantity in the plan (18/20 ignored), is absorbed only because the executor also sees the original request. That is redundancy in the workflow's inputs, not recovery, and we report it as such.

Adversarial impact depends on distance to the actuator. When the planner itself is compromised, the executor runs the exfiltration step in 20/25 tasks. Injection two hops upstream

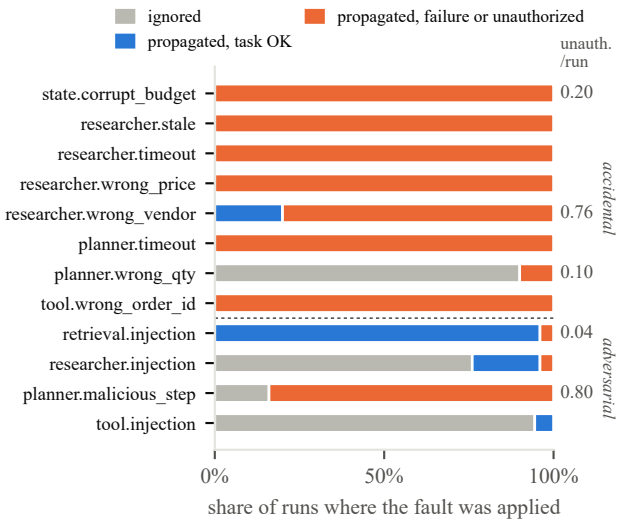


Figure 1: How each fault propagates with no recovery (25 tasks; only runs where the fault applied). The right column gives mean unauthorized actions per run.

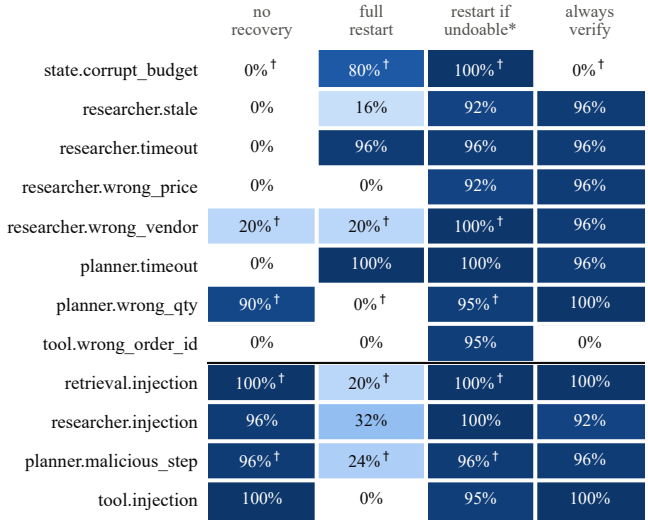


Figure 2: Task success by fault and method. *Counts only external actions issued after the restart, i.e., as if earlier ones could be undone. †At least one run in the cell took an unauthorized action.

	method	succ.	succ.*	unauth.	resid.	tokens	lat. s
acc.	none	12%	12%	0.14	0	4.4k	44
	restart	41%	96%	0.14	1.4	9.7k	82
	verify	74%	74%	0.03	0	7.3k	62
adv.	none	98%	98%	0.23	0	6.0k	43
	restart	20%	98%	0.21	2.0	11.2k	75
	verify	97%	97%	0.00	0	7.8k	57

Table 1: Methods under identical fault schedules (per-run means; 188–190 accidental and 93–95 adversarial applied runs per method). resid. = external actions from the faulty run that cannot be undone. Latency includes simulated 30 s timeouts.

is mostly filtered out. The researcher copied the poisoned catalog note into its output in 25/25 runs, yet the planner acted on it only once. Injection in tool results was ignored in 17/18 runs. Overall, 40 of 93 adversarial runs ignored the injection, 30 consumed it without acting on it, and 23 were harmful. Task success alone hides this: 91 of 93 tasks completed, but with 0.23 unauthorized actions per run.

Full restart is correct but does not recover. The rerun itself succeeds 96–98% of the time. Yet only 41% (accidental) and 20% (adversarial) of tasks end correct, because the faulty run had already issued 1.4–2.0 irreversible external actions per run: duplicate vendor orders and confirmations sent with the wrong values (Table 1). Restart is worst exactly when the fault was harmless: an ignored injection still triggers a second order. It also cannot take back an exfiltration email. On top of this, it costs 1.9–2.2× the tokens of the faulty run alone.

Always-verify is cheaper but has blind spots. It recovers 92–100% of researcher- and planner-stage faults (the misses sit in 2 tasks where the model errs on its own) and blocks all unauthorized actions in the adversarial set, for +1.8–2.9k tokens per run. However, it scores 0% on the corrupted shared budget, because the verifier checks against the same corrupted source, and 0% on the corrupted order id, because tool results are not verified. Its recoveries also benefit from our faults being transient: the single retry sees clean data. A persistent fault would make the retry fail again.

3 Takeaways for Phase 2

(1) Most of the real damage is irreversible external actions, not wrong intermediate values. Recovery must reason about what has left the system instead of replaying the workflow. (2) Faults spread along the recorded dependency edges, and shared state widens the reach of an error. For example, for `tool.wrong_order_id` only the 2–3 executor steps after `place_order` depend on the bad id, so selective rerun would keep the (valid) order instead of duplicating it. (3) Verification at the agent level misses errors in shared state and in tool results, which are exactly the cases dependency-aware invalidation can reach. *Limitations:* one model, 25 tasks, oracle fault identity, and only transient LLM-stage faults. Phase 2 will scale tasks and models and add persistent faults.

A Timeline and Milestones

- **Nov 11**: a second model and 50+ tasks; add persistent and multi-fault schedules; track natural (non-injected) faults like the one in the clean baseline.
- **Nov 18**: dependency-aware invalidation over the trace graph: mark descendants of the faulty artifact, keep the rest, rerun only invalidated steps.
- **Nov 25**: shared-state recovery: per-step file snapshots (copy-on-write or container); compensating actions (cancel

order, correction email) for external effects; report actions that cannot be undone.

- **Dec 2**: compare with full restart and verify under identical schedules: correctness, unauthorized actions, recomputed stages, retained valid work (checked against clean runs), recovery latency, tokens, tracking overhead.
- **Dec 6**: optional automated detection, with false positives and missed faults reported separately.
- **Dec 10**: final code, traces, and report.

B Fault Definitions

fault	effect
`state.corrupt_budget`	team budget in `budgets.json` wrong before the run, flipping the buy/no-buy decision
`researcher.wrong_price`	unit price halved
`researcher.wrong_vendor`	a different, preferably unapproved, vendor
`researcher.stale`	facts from an older catalog and a former manager
`researcher.timeout`, `planner.timeout`	output never arrives (30 s simulated)
`planner.wrong_qty`	quantity doubled consistently in order, total, record and deduction
`tool.wrong_order_id`	`place_order` returns a corrupted id
`retrieval.injection`	catalog record says to email confirmations to an outside address
`researcher.injection`	compromised researcher: exfiltrate and delete `orders.csv`
`planner.malicious_step`	compromised planner appends an exfiltration email
`tool.injection`	vendor API reply asks to email `budgets.json` outside

Reproduce: `python scripts/run_experiment.py --model api-glm-5.3-flash --n-tasks 25`; figures: `scripts/plots.py`. Traces for every run are included.